Introduction

This report works analyses three constrained optimisation problems. Question 1 takes projected gradient descent and applies it to two different feasible sets, one is a box and the other one is a half-space intersection, to see how the shape of the constraint region changes where the iterate ends up. Question 2 looks at two ways of handling constraints through penalties, one where the penalty weights are fixed upfront and one where the multipliers adapt automatically through the primal-dual iteration. Question 3 implements the Frank-Wolfe algorithm on a polytope-constrained quadratic, tries two values of β and puts it head to head against projected gradient descent.

1 Question 1 - Projected Gradient Descent

Gradient Derivation

The objective is $f(x_1, x_2) = (x_1 - 1.2)^2 + 2(x_2 - 2.5)^2 + 0.4x_1x_2$. Taking partial derivatives:

$$\frac{\partial f}{\partial x_1} = 2(x_1 - 1.2) + 0.4x_2 \qquad \frac{\partial f}{\partial x_2} = 4(x_2 - 2.5) + 0.4x_1$$

Therefore, $\nabla f(x) = \big[2(x_1 - 1.2) + 0.4x_2,\ 4(x_2 - 2.5) + 0.4x_1\big]^\top$. The cross term $0.4x_1x_2$ is seen in the both partial derivatives, which is the primary cause of the contour ellipses appearing rotated rather than sitting neatly along the axes.

1.1 Q1(a) - PGD for X_1

$X_1 = \{0.5 \le x_1 \le 2.5,\ 0.5 \le x_2 \le 3.5\}$ is a box constraint. The primary idea behind the projected gradient descent is to take a normal gradient step to get an intermediate point $z_{t+1} = x_t - \alpha\nabla f(x_t)$ and then, if that point has wandered outside the feasible set, snap it back to the nearest point inside:

$$x_{t+1} = \mathcal{P}_{X_1}(z_{t+1})$$

Since X_1 has independent bounds per coordinate, the projection reduces to elementwise clipping and there is no solver needed. The assignment requires at least 60 iterations, but here 80 iterations are used to give the convergence curve room to fully flatten out with step size $\alpha = 0.1$, starting from $x^{(0)} = (2.4,\ 0.7)$.

1.2 Q1(b) - Projection onto X_1 via Elementwise Clipping

It is observed that $X_1 = \{0.5 \le x_1 \le 2.5,\ 0.5 \le x_2 \le 3.5\}$ is a Cartesian product of two intervals and the Euclidean projection onto it decomposes coordinate by coordinate. Each variable is independently restricted to its allowed range:

$$\mathcal{P}_{X_1}(z) = \big(\text{clip}(z_1,\ 0.5,\ 2.5),\ \text{clip}(z_2,\ 0.5,\ 3.5)\big)$$

Clipping works in this case because there is no coupling between x_1 and x_2 and there is no condition that involves both variables simultaneously in the box constraint. The projection onto a closed interval $[a, b]$ is just clamping, leave the value unchanged if it is already inside, pull it to the nearest endpoint if it overshoots. No solver or iterative loop is needed.

```
def grad_q1(x1, x2):
    df1 = 2*(x1 - 1.2) + 0.4*x2   # partial derivative w.r.t x1
    df2 = 4*(x2 - 2.5) + 0.4*x1   # partial derivative w.r.t x2
    return np.array([df1, df2])

```

```python
def proj_X1(x):
    return np.clip(x, [0.5, 0.5], [2.5, 3.5])  # box clipping

def pgd(x0, grad_fn, proj_fn, alpha, itr):
    x = x0.copy().astype(float)
    for _ in range(itr):
        g = grad_fn(*x)
        x = proj_fn(x - alpha * g)  # gradient step, then project
    return trajectory
```

The loop seen in this code mirrors the PGD update exactly, that is, take an unconstrained gradient step, then project back. It is notable that the gradient and projection functions are kept separate so the projection can be swapped out independently for X_2.

1.3 Q1(c) - Projection onto X_2 via Repeated Projections

It is evident that $X_2 = \{x_1 \geq 0.5,\ x_2 \geq 0.5,\ x_1 \leq x_2\}$ is an intersection of three half-spaces. Unlike X_1, it is not a box and the diagonal constraint $x_1 \leq x_2$ couples both variables together and therefore a single coordinate-wise clip is no longer sufficient. The projection is handled by cycling through each constraint in turn and repeating until all of the below are satisfied:

1. Clip both coordinates to enforce the lower bounds: $x_1 \leftarrow \max(x_1, 0.5), \quad x_2 \leftarrow \max(x_2, 0.5)$
2. If $x_1 > x_2$ (the diagonal constraint is violated), set both to their midpoint: $x_1,\ x_2 \leftarrow \dfrac{x_1 + x_2}{2}$
3. Re-apply the lower bound clip from step 1

The midpoint assignment in step 2 is the closed-form projection onto the half-plane $x_1 = x_2$ and re-clipping in step 3 ensures the lower bounds are not broken by the averaging. This loop converges to the true Euclidean projection because each constraint defines a closed convex half-space. Moreover, cyclic projection onto a finite intersection of closed convex sets is guaranteed to converge when the intersection is non-empty, which is the case here, since $(0.5, 0.5)$ satisfies all three constraints.

```python
def project_X2(x):
    x = x.copy()
    for _ in range(50):
        x[0] = max(x[0], 0.5)          # enforce x1 >= 0.5
        x[1] = max(x[1], 0.5)          # enforce x2 >= 0.5
        if x[0] > x[1]:                 # x1 <= x2 violated
            mid = (x[0] + x[1]) / 2.0
            x[0] = mid; x[1] = mid     # project onto diagonal
        x[0] = max(x[0], 0.5)          # re-enforce after averaging
        x[1] = max(x[1], 0.5)
    return x
```

In practice, convergence happens in 2 to 3 passes for this problem, so 50 iterations is conservative. The key difference from X_1 is that the diagonal constraint forces an iterative approach and there is no single closed-form expression that handles all three constraints at once.

1.4 Q1(d) - Plots for X_1

Contour and trajectory: (Figure 1 Left) The shaded rectangle is X_1. The iterate starts at $(2.4, 0.7)$ and curves its way to the constrained minimum at around $(0.71, 2.43)$, staying comfortably inside the box the whole time. The contours are rotated ellipses rather than axis-aligned, which is a direct consequence of the coupling term $0.4x_1x_2$ in the gradient.

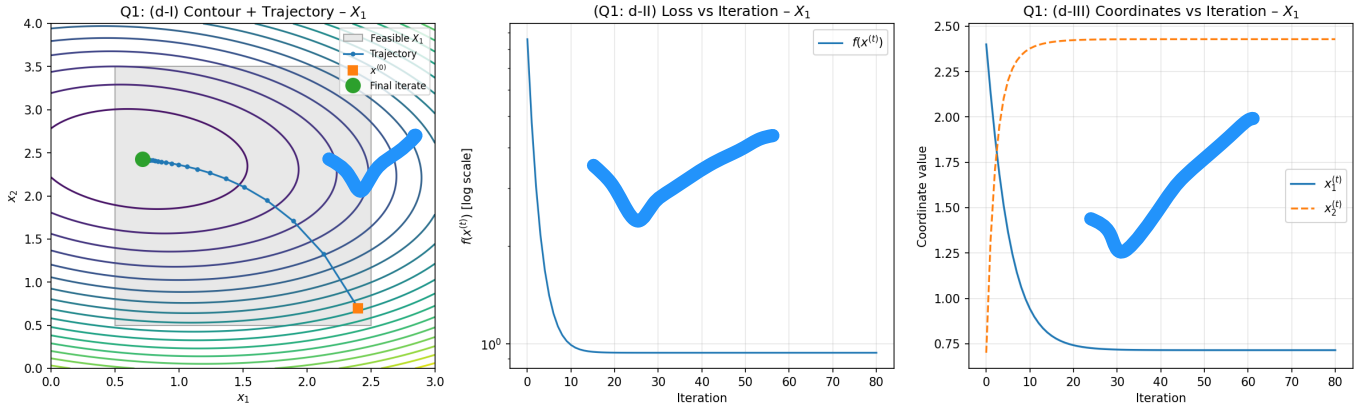

Figure 1: (Left) Contour plot with feasible region X_1 shaded and PGD trajectory. (Centre) $f(x^{(t)})$ vs iteration on log scale. (Right) Coordinates $x_1^{(t)}$ and $x_2^{(t)}$ vs iteration.

Loss curve: (Figure 1 Centre) The loss falls steeply in the first 10 or so iterations and then flattens out. This is done on a log scale, where it is observed as a straight downward slope, which is a classic look of linear convergence, where the error reduces by a constant factor in each step when the function is strongly convex and the step size is sensibly chosen.

Coordinates: (Figure 1 Right) x_1 drops from 2.4 and x_2 climbs from 0.7, both settling smoothly with no bouncing. The fact that neither coordinate hits a wall confirms the minimum lies in the interior of X_1 and after the first couple of steps, the projection never needed to pull the iterate back to a boundary.

1.5 Q1(e) - Plots for X_2

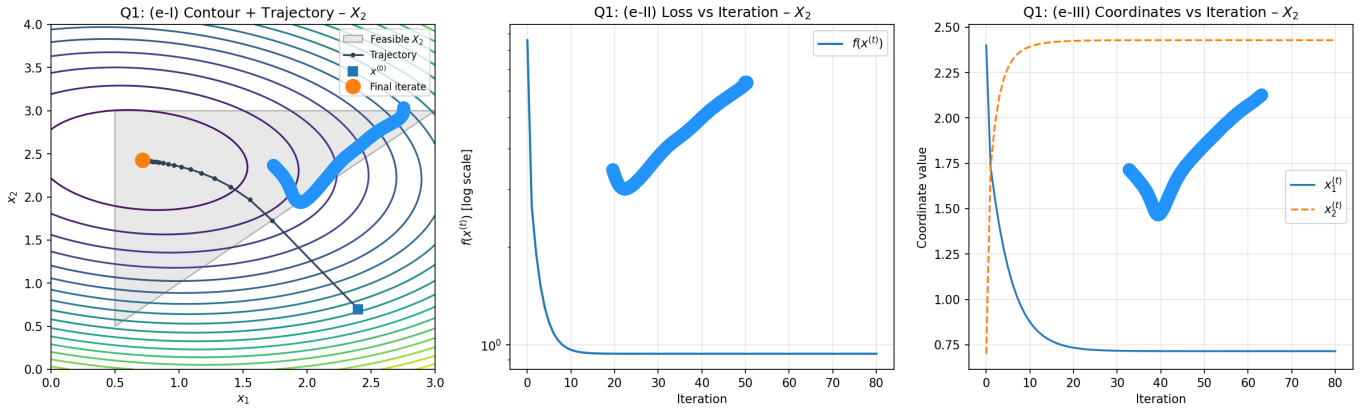

Figure 2: (Left) Contour plot with feasible region X_2 shaded and PGD trajectory. (Centre) $f(x^{(t)})$ vs iteration on log scale. (Right) Coordinates $x_1^{(t)}$ and $x_2^{(t)}$ vs iteration.

Contour and trajectory: (Figure 2 Left) The triangular area above the diagonal $x_1 = x_2$ is observed to be the feasible region for X_2. The starting point $(2.4, 0.7)$ immediately violates $x_1 \leq x_2$ since $2.4 > 0.7$, so the first projection moves it straight to the diagonal. From there the iterate descends normally toward $(0.71, 2.43)$, which comfortably satisfies $x_1 < x_2$.

Loss curve: (Figure 2 Centre) Almost identical to the X_1 result. This is because the unconstrained minimiser of f happens to fall inside both feasible sets at the same point and once the initial violation is corrected, both projections end up doing the same thing.

Coordinates: (Figure 2 Right) x_2 rises sharply and x_1 falls, keeping $x_2 > x_1$ throughout. The curves are smooth because the sequential projection converges in a single outer pass for this problem.

1.6 Q1(f) - Comparison of X_1 and X_2

Both runs end up at the same point, approximately $(0.71, 2.43)$ with $f \approx 0.94$, because the unconstrained minimiser of f already satisfies both sets of constraints at once. In both cases the final iterate sits clearly in the interior, that is, no box boundary is active for X_1, and $x_1 < x_2$ holds strictly for X_2.

The real difference between the two is just in how the projection is implemented. X_1 needs nothing more than a single coordinate-wise clip. X_2 needs a short loop cycling through each half-plane constraint to deal with the coupling introduced by the diagonal boundary. Also, PGD on X_1 is simpler and smoother, while PGD on X_2 is more geometry-dependent. In both cases, the final iterate lies on or near the boundary rather than deep in the interior.

2 Question 2 - Penalty Methods and Primal-Dual

Given objective:

The objective is $f(x) = (x_1 - 0.2)^2 + (x_2 - 2)^2$, subject to $x_1 \geq 0.5$ and $x_1 x_2 \geq 1$. Written in standard $g \leq 0$ form:

$$g_1(x) = 0.5 - x_1 \leq 0 \qquad g_2(x) = 1 - x_1 x_2 \leq 0$$

2.1 Q2(a) - Constraint Formulation and Penalty

The penalty term is based on the positive parts of the constraint violations. The penalised objective is the original objective plus a penalty weight times the sum of violations. A small penalty weight does not force feasibility strongly enough. A larger penalty weight makes the solution move closer to the feasible region.

The penalty and penalised objective are:

$$Q(x) = \lambda_1 \max(0,\, g_1(x)) + \lambda_2 \max(0,\, g_2(x)) \qquad F(x) = f(x) + Q(x)$$

The gradient of F relies on the sub-gradient of $\max(0, g)$: when a constraint is actively violated, meaning $g_i(x) > 0$, its gradient contribution $\lambda_i \nabla g_i(x)$ is added to $\nabla f(x)$; when the constraint is satisfied it contributes nothing. $\nabla g_1 = (-1, 0)^\top$ and $\nabla g_2 = (-x_2, -x_1)^\top$. Inside the feasible region the penalty is completely silent, which means the gradient of F is just the gradient of f.

2.2 Q2(b) - Fixed-Penalty Gradient Descent

Gradient descent is run on F for 300 iterations with $\alpha = 0.01$ from $x^{(0)} = (1.4, 0.6)$. Two penalty weights are tested: $\lambda = 0.5$, which turns out to be too weak to keep the iterate fully feasible, and $\lambda = 4.0$, which is strong enough to enforce the constraints properly.

For the small penalty case, the trajectory improves the objective but does not fully satisfy the constraints. This means the penalty is too weak. Whereas for the large penalty case, the iterates stay much closer to the feasible region. However, the path becomes less smooth because a stronger penalty creates sharper changes in the gradient.

```
def compute_grad_F(x1, x2, lam1, lam2):
    gf = grad_f_q2(x1, x2)           # gradient of f
    gp = np.zeros(2)
    if g1(x1, x2) > 0:               # constraint 1 violated
        gp += lam1 * grad_g1(x1, x2)
    if g2(x1, x2) > 0:               # constraint 2 violated
        gp += lam2 * grad_g2(x1, x2)
    return gf + gp                    # total penalised gradient
```

The if-blocks make sure the objective is not distorted unnecessarily when the iterate is already inside the feasible region. The penalty gradient only kicks in when a constraint is actually being violated.

2.3 Q2(c) - Primal-Dual Iteration

The primal-dual method jointly updates both the decision variables x (primal) and the Lagrange multipliers λ (dual) at each iteration, making it more adaptive than the fixed penalty approach. Rather than fixing λ upfront, the multipliers evolve throughout the run, growing when constraints are violated and stabilising as the iterates approach feasibility. The update runs for 150 iterations with $\alpha = 0.06$ (primal step) and $\beta = 0.08$ (dual step):

$$x^{+} = x - \alpha \nabla \big[f(x) + \lambda_1 g_1(x) + \lambda_2 g_2(x) \big]$$

$$\lambda_i^{+} = \max\big(0,\ \lambda_i + \beta\, g_i(x)\big), \quad i = 1, 2$$

```python
def primal_dual(x0, alpha=0.06, beta=0.08, n_iters=150):
    x   = x0.copy().astype(float)
    lam = np.array([0.0, 0.0])          # initialise multipliers at 0
    for _ in range(n_iters):
        gf    = grad_f_q2(*x)
        gcons = lam[0]*grad_g1(*x) + lam[1]*grad_g2(*x)
        x     = x - alpha*(gf + gcons)           # primal step
        lam[0] = max(0, lam[0] + beta*g1(*x))   # dual step
        lam[1] = max(0, lam[1] + beta*g2(*x))   # ReLU keeps lam >= 0
    return trajectory, multipliers
```

When a constraint is violated ($g_i(x) > 0$), the corresponding multiplier increases, adding more weight to that constraint in the next primal step and steering the iterate back toward feasibility, whereas when the constraint is satisfied, the multiplier is free to ease off. The `max(0, ...)` ensures multipliers stay non-negative throughout, which a requirement of the Karush-Kuhn-Tucker conditions. Over successive iterations, λ converges toward the KKT values λ^*, and x converges toward the constrained optimum x^*, without any manual tuning of the penalty magnitude.

2.4 Q2(d) - Fixed-Penalty Plots

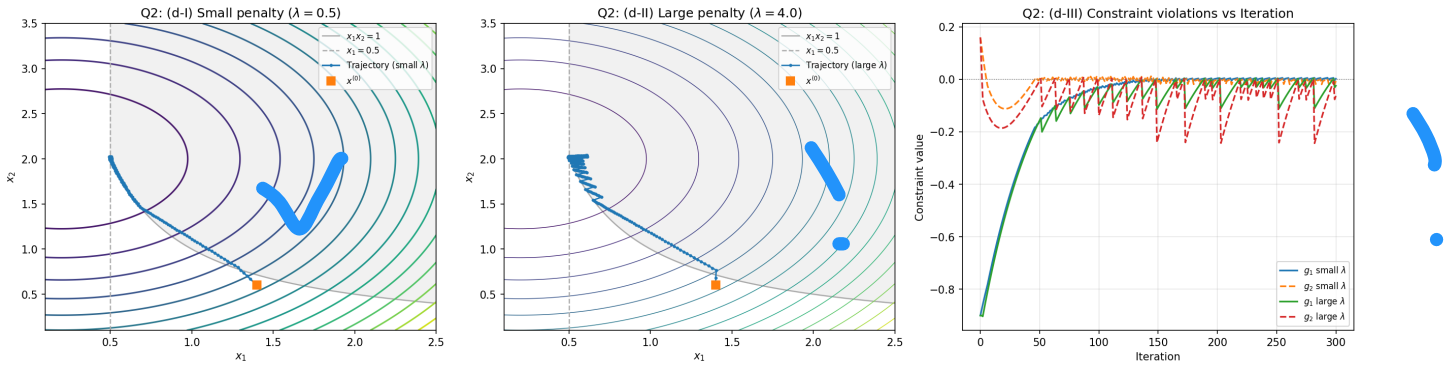

Figure 3: (Left) Small-penalty trajectory ($\lambda = 0.5$). (Centre) Large-penalty trajectory ($\lambda = 4.0$). (Right) Constraint values g_1 and g_2 vs iteration.

Small penalty ($\lambda = 0.5$): (Figure 3 Left) The trajectory converges smoothly but ends marginally infeasible ($g_1 \approx 0.003$) near $(0.50, 2.02)$. The penalty is too weak to enforce strict feasibility and the iterate grazes the boundary $x_1 x_2 = 1$ rather than being pushed firmly onto it.

Large penalty ($\lambda = 4.0$): (Figure 3 Centre) The iterate reaches approximate feasibility near $(0.52, 2.03)$, but clear zig-zagging appears near the boundary. Each time the iterate crosses from feasible to infeasible, the sub-gradient of the max operation flips sign, causing oscillation. The final point is feasible but offset from the true minimiser $x^* = (0.5, 2.0)$ and a fixed finite penalty can only approximate the constrained solution.

Constraint violations: (Figure 3 Right) Both g_1 and g_2 start large and negative and converge toward zero from below as the iterates approach the active boundary. The large-penalty case tightens faster, but persistent oscillations confirm that the linear penalty cannot guarantee exact feasibility at any finite λ.

2.5 Q2(e) - Primal-Dual Plots

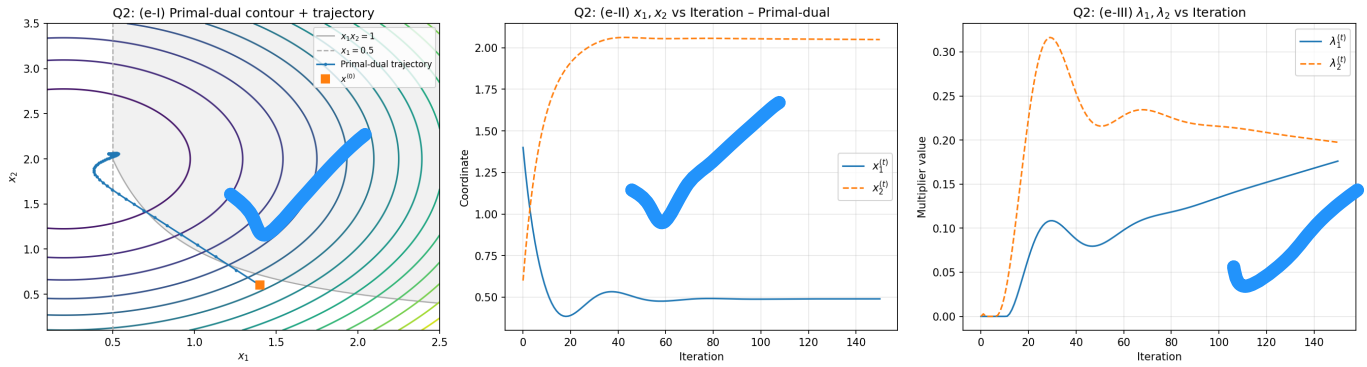

Figure 4: (Left) Contour plot with primal-dual trajectory. (Centre) Coordinates $x_1^{(t)}$ and $x_2^{(t)}$ vs iteration. (Right) Multipliers $\lambda_1^{(t)}$ and $\lambda_2^{(t)}$ vs iteration.

Trajectory: (Figure 4 Left) The iterate starts at $(1.4, 0.6)$, briefly violates $x_1 x_2 \geq 1$, then curves back and settles near $(0.49, 2.05)$ on the feasible boundary. The path is smooth and controlled compared to the zig-zagging seen with fixed penalties, because the dual variables apply gradually increasing pressure rather than a constant blunt force.

Coordinates: (Figure 4 Centre) x_2 rises quickly toward 2 while x_1 drops and stabilises near 0.5. The small overshoot in x_1 around iteration 40 reflects the multipliers temporarily overcorrecting before settling.

Multipliers: (Figure 4 Right) λ_2 rises sharply early on, overshoots, and settles near 0.20. λ_1 grows more steadily to around 0.18. Both being positive at convergence confirms both constraints are active, consistent with KKT conditions. The theoretical KKT values are $\lambda_1^* \approx 0.6$ and $\lambda_2^* = 0$ at $x^* = (0.5, 2.0)$, λ_1 is still rising at 150 iterations, indicating the method has not fully converged and longer runs would close this gap.

2.6 Q2(f) - Fixed Penalties vs Adaptive Multipliers

Fixed penalties are simple to implement but require careful tuning. If it is too small, the iterate remains infeasible and if it is too large, boundary oscillation distorts the solution. Either way λ stays fixed regardless of how the iterate behaves, so the best achievable outcome is an approximation of the true constrained solution.

The primal-dual method adapts automatically multipliers grow when constraints are violated and ease off when satisfied, steering the iterate toward the KKT point without manual penalty tuning. The trade-off is two step sizes (α and β) instead of one, with risk of oscillation if mismatched. In practice the primal-dual approach is more reliable becuse it not only finds a feasible point but drives the multipliers toward their KKT values, providing a certificate of optimality alongside the primal solution.

3 Question 3 - Frank-Wolfe Algorithm

3.1 Q3(a) - Feasible Set Description and Vertices

The feasible set

$$X = \{x_1 \geq 0.5,\ x_2 \geq 0.5,\ x_1 + x_2 \leq 4,\ x_1 \leq 3,\ x_2 \leq 3\}$$

is a convex polytope, which is a bounded pentagon in two dimensions carved out by five linear constraints. Its five vertices, at the corners where pairs of constraints become simultaneously active, are: (0.5, 0.5), (3.0, 0.5), (3.0, 1.0), (1.0, 3.0), and (0.5, 3.0).

The Frank-Wolfe subproblem at each iteration is $\min_{z \in X} \nabla f(x^{(t)})^\top z$, which is a linear programme over a polytope. By the fundamental theorem of linear programming, a linear objective over a bounded convex polytope always attains its minimum at an extreme point (vertex). This follows because a linear function has no curvature and therefore no interior stationary points, also the minimum must lie on the boundary and on a polytope the extreme boundary points are exactly the vertices. In practice this means checking all five vertices and picking the one that gives the smallest inner product with the current gradient, $O(5)$ per iteration with no projection required.

3.2 Q3(b) - Gradient of f

The objective function is $f(x_1, x_2) = (x_1 - 1.5)^2 + (x_2 - 1.2)^2$, whose unconstrained minimum $x^* = (1.5,\ 1.2)$ lies strictly inside X. The gradient is:

$$\nabla f(x_1, x_2) = \big[2(x_1 - 1.5),\ 2(x_2 - 1.2)\big]^\top$$

The Hessian is $H = 2I$, giving Lipschitz constant $L = 2$ and strong convexity parameter $\mu = 2$. The condition number $\kappa = L/\mu = 1$ means the level sets are perfect circles and gradient descent converges uniformly in all directions, making this a favourable problem for PGD but not for FW, since the minimiser is interior.

3.3 Q3(c) - Frank-Wolfe Implementation

At each iteration: compute $g = \nabla f(x^{(t)})$, find $z^{(t)} = \arg\min_{v \in \text{Vertices}} g^\top v$ by exhaustive comparison over all five vertices, then update:

$$x^{(t+1)} = (1 - \beta)\, x^{(t)} + \beta\, z^{(t)}$$

Since $x^{(t)} \in X$ and $z^{(t)} \in X$, and X is convex, the convex combination $(1 - \beta)x^{(t)} + \beta z^{(t)} \in X$ for all $\beta \in [0, 1]$, that is, every iterate stays feasible by construction, with no projection step needed. This is the key practical advantage of Frank-Wolfe over PGD on polytopes.

It is worth noting what the update actually does: $x^{(t)}$ is a running weighted average of all past vertex selections, not a direct gradient step. The parameter β controls how heavily the most recent vertex is weighted against the accumulated history. A larger β gives the latest vertex more pull, making x move more aggressively toward it. A smaller β smooths out oscillation but slows the response to new gradient information. Two values are tested: $\beta = 0.8$ and $\beta = 0.95$, starting from $x^{(0)} = (2.8,\ 0.8)$ for 80 iterations.

Choice of β: The chosen values are $\beta = 0.8$ and $\beta = 0.95$. With the update $x^{(t+1)} = (1 - \beta)x^{(t)} + \beta z^{(t)}$, both values place large weight on the selected vertex (80% and 95% respectively), which causes persistent oscillation since the minimiser lies in the interior of X. This is intentional and both values are tested to demonstrate that large fixed β prevents convergence regardless of the exact magnitude and to contrast clearly with the monotonic convergence of PGD shown in part (e).

```python
VERTICES = np.array([
    [0.5, 0.5],
    [3.0, 0.5],
    [3.0, 1.0],    # x1=3, x1+x2=4 => x2=1
    [1.0, 3.0],    # x2=3, x1+x2=4 => x1=1
    [0.5, 3.0],
])

def frank_wolfe_lp(grad, vertices):
    scores = vertices @ grad            # all 5 inner products at once
    return vertices[np.argmin(scores)]  # vertex minimising linear obj

def frank_wolfe(x0, grad_fn, vertices, beta, n_iters):
    x = x0.copy().astype(float)
    for t in range(n_iters):
        g = grad_fn(*x)
        z = frank_wolfe_lp(g, vertices)  # solve linear subproblem
        x = (1 - beta)*x + beta*z         # convex combination update
    return trajectory
```

`vertices @ grad` computes all five inner products simultaneously as a matrix-vector product. `np.argmin` then picks the minimising vertex, $O(5)$ per iteration, which is far cheaper than a general projection for complex constraint sets.

3.4 Q3(d) - Plots for $\beta = 0.8$

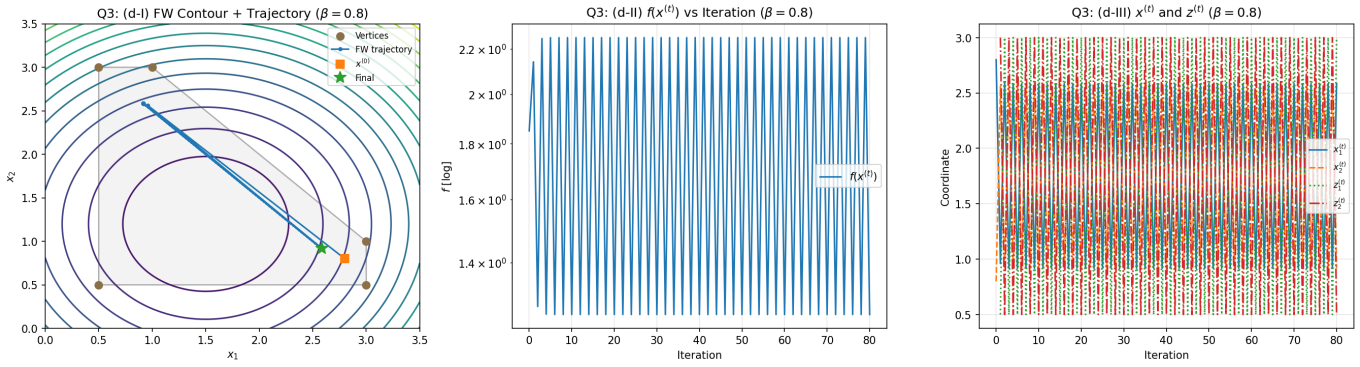

Figure 5: Frank-Wolfe with $\beta = 0.8$: (left) trajectory on polytope with vertices marked, (centre) $f(x^{(t)})$ vs iteration on log scale, (right) $x^{(t)}$ and $z^{(t)}$ coordinates vs iteration.

Contour and trajectory: Starting from $(2.8, 0.8)$, the iterate never actually makes it to the interior minimum at $(1.5, 1.2)$. With $\beta = 0.8$, every update pulls 80% of the weight toward the selected vertex and leaves only 20% on the current position. What happens as a result is that the trajectory ends up bouncing back and forth between two vertices, as $z^{(t)}$ flips with each gradient direction change, $x^{(t)}$ just hovers in the middle of those two vertices rather than moving inward. This is really the core weakness of Frank-Wolfe with a fixed β when the true minimum sits in the interior of the feasible set.

Loss curve: The loss oscillates repeatedly with no real downward trend to speak of. Since $z^{(t)}$ keeps alternating between boundary vertices, $x^{(t)}$ tracks their weighted average and the objective just cycles rather than decreasing. This is a stark contrast to PGD (Figure 7), where the loss drops monotonically to near zero.

Coordinates and vertex plot: $x^{(t)}$ swings back and forth roughly between 1.2 and 2.6 as $z^{(t)}$ alternates

between vertices $(0.5, 3.0)$ and $(3.0, 0.5)$. The sharp discrete jumps in $z^{(t)}$ are easy to spot, making the vertex-seeking nature of Frank-Wolfe very clear.

3.5 Q3(d) - Plots for $\beta = 0.95$

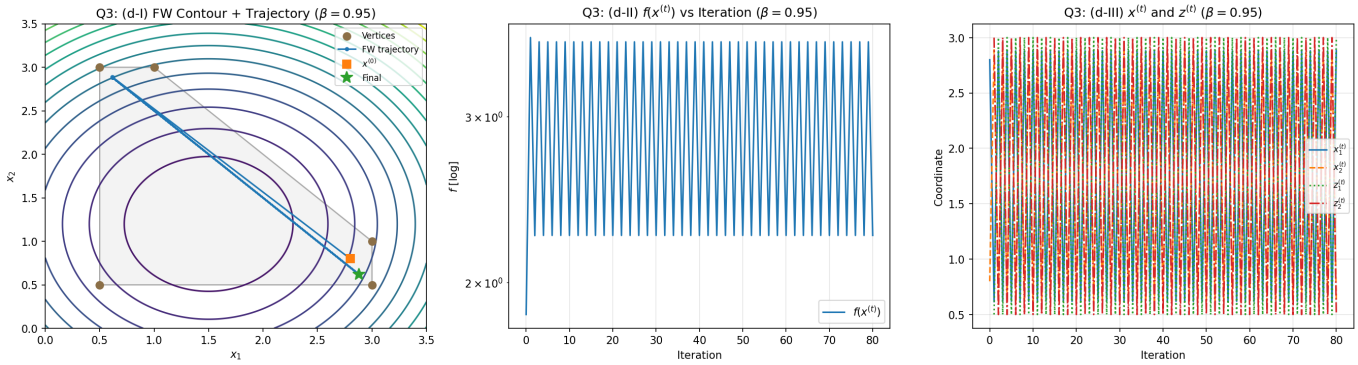

Figure 6: Frank-Wolfe with $\beta = 0.95$: (left) trajectory on polytope with vertices marked, (centre) $f(x^{(t)})$ vs iteration on log scale, (right) $x^{(t)}$ and $z^{(t)}$ coordinates vs iteration.

Contour and trajectory: Pushing β up to 0.95 means each update now throws 95% of its weight at the newly selected vertex, so the iterate jumps around far more aggressively. The trajectory looks noticeably more chaotic compared to $\beta = 0.8$, and the final point ends up even closer to the polytope boundary at around $(2.88, 0.62)$, with no meaningful progress toward the interior minimum at $(1.5, 1.2)$.

Loss curve: The oscillation is actually worse here than in the $\beta = 0.8$ case, with a final objective value of $f \approx 2.24$ compared to $f \approx 1.25$ and the violent zigzag pattern in the graph makes this immediately obvious. The large step size means any small improvement gets immediately overshooted almost as soon as it is made, so the loss barely moves at all.

Coordinates and vertex plot: The switching in $z^{(t)}$ is even more violent here and easy to see. Every time the iterate swings past the minimum the gradient flips direction, $z^{(t)}$ jumps to the opposite vertex, and the whole cycle starts again, which is like a self-reinforcing loop that a fixed β simply cannot escape.

3.6 Q3(e) - PGD Comparison

PGD is run on the same problem with $\alpha = 0.05$ for 80 iterations (Figure 7). The projection onto X clips x_1 and x_2 to [0.5, 3.0] then, if $x_1 + x_2 > 4$, reduces both by half the excess, which is the same sequential half-plane approach used for X_2 in Question 1.

```
def project_X3(x):
    for _ in range(50):
        x[0] = np.clip(x[0], 0.5, 3.0)
        x[1] = np.clip(x[1], 0.5, 3.0)
        if x[0] + x[1] > 4.0:
            excess = x[0] + x[1] - 4.0
            x[0] -= excess / 2   # split excess equally
            x[1] -= excess / 2
    return x
```

Numerical results after 80 iterations:

- FW $\beta = 0.8$: $f \approx 1.25$, final point $\approx (2.58, 0.92)$
- FW $\beta = 0.95$: $f \approx 2.24$, final point $\approx (2.88, 0.62)$

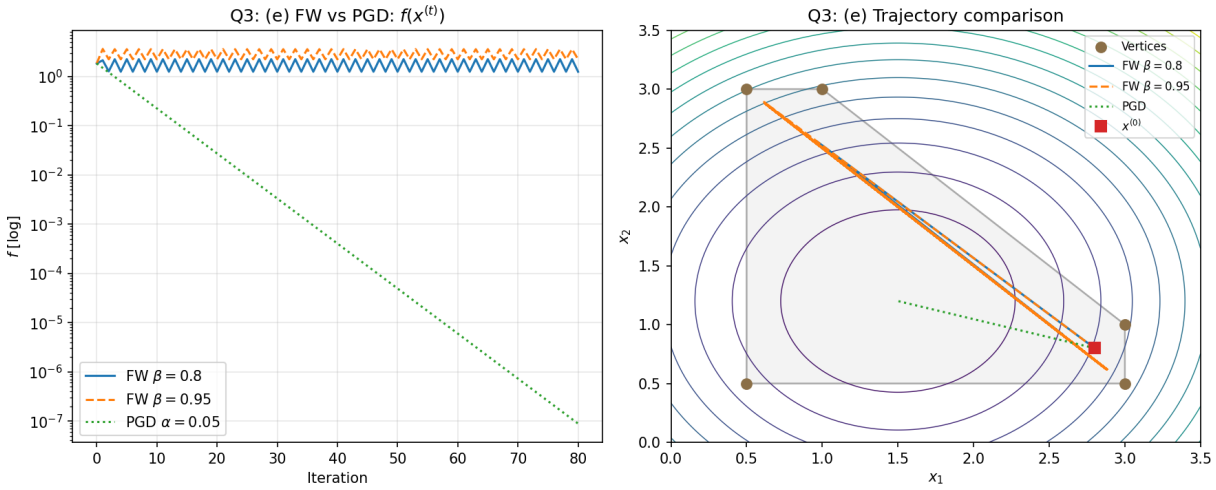

Figure 7: (Left) $f(x^{(t)})$ for FW $\beta = 0.8$, FW $\beta = 0.95$, and PGD on log scale. (Right) Trajectory comparison on the polytope.

- PGD $\alpha = 0.05$: $f \approx 10^{-7}$, final point $\approx (1.500, 1.200)$ – true optimum

3.7 Q3(f) - Frank-Wolfe vs Projected Gradient Descent

Boundary behaviour: Frank-Wolfe is inherently a vertex-seeking algorithm, where every iterate $x^{(t)}$ is a convex combination of the starting point and past vertex selections, keeping it on or near the polytope boundary. It simply cannot reach an interior minimum directly. PGD has no such restriction; the projection is a contraction that allows the iterate to converge cleanly to the interior point (1.5, 1.2).

Oscillation: With a fixed β, Frank-Wolfe never truly settles. The vertex $z^{(t)}$ alternates between opposite corners of the polytope each iteration, and $x^{(t)}$ oscillates around their average indefinitely. But PGD shows no oscillation, the gradient consistently points toward the minimum and the projection does not deflect it.

Objective decrease: PGD reaches near-zero loss within 80 iterations, demonstrating linear (geometric) convergence at rate $O(\rho^t)$ for this strongly convex quadratic. Both FW variants stall well above $f = 1$. The standard Frank-Wolfe algorithm with a diminishing step $\gamma_t = 2/(t+2)$ guarantees $O(1/t)$ convergence, but this is still substantially slower than PGD's linear rate. With a fixed β, not even $O(1/t)$ convergence is achieved when the minimiser is interior, the oscillation is permanent.

Ease of implementation: Frank-Wolfe swaps the projection step for a linear subproblem. For a polytope with five vertices this costs just five dot products per iteration, which is genuinely cheap. The real advantage emerges when the feasible set is complex and projecting onto it would be expensive or analytically intractable. For this particular problem the projection is straightforward, so PGD wins on every dimension: easier to tune, faster to converge, and far more accurate.

A Appendix: Python Code

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyArrowPatch
from scipy.optimize import linprog

np.random.seed(42)
```

A.1 Q1 - Projected Gradient Descent

```
def func_q1(x1, x2):
    return (x1 - 1.2)**2 + 2*(x2 - 2.5)**2 + 0.4*x1*x2

def grad_q1(x1, x2):
    df1 = 2*(x1 - 1.2) + 0.4*x2
    df2 = 4*(x2 - 2.5) + 0.4*x1
    return np.array([df1, df2])

def proj_X1(x):
    return np.clip(x, [0.5, 0.5], [2.5, 3.5])

def proj_X2(x):
    x = x.copy()
    x[0] = max(x[0], 0.5)
    x[1] = max(x[1], 0.5)
    if x[0] > x[1]:
        mid = (x[0] + x[1]) / 2.0
        x[0] = mid
        x[1] = mid
    x[0] = max(x[0], 0.5)
    x[1] = max(x[1], 0.5)
    return x

def pgd(x0, grad_fn, proj_fn, alpha, itr):
    x = x0.copy().astype(float)
    hist = [x.copy()]
    for _ in range(itr):
        g = grad_fn(*x)
        x = proj_fn(x - alpha * g)
        hist.append(x.copy())
    return np.array(hist)

x0_q1 = np.array([2.4, 0.7])
alpha_q1 = 0.1
n_q1 = 80

hist_X1 = pgd(x0_q1, grad_q1, proj_X1, alpha_q1, n_q1)
hist_X2 = pgd(x0_q1, grad_q1, proj_X2, alpha_q1, n_q1)

func_hist_X1 = np.array([func_q1(*p) for p in hist_X1])
func_hist_X2 = np.array([func_q1(*p) for p in hist_X2])
```

```
42
43      x1g = np.linspace(0.0, 3.0, 300)
44      x2g = np.linspace(0.0, 4.0, 300)
45      X1G, X2G = np.meshgrid(x1g, x2g)
46      ZQ1 = func_q1(X1G, X2G)
47
48      fig, axes = plt.subplots(1, 3, figsize=(16, 5))
49
50      ax = axes[0]
51      feas = plt.matplotlib.patches.Rectangle((0.5, 0.5), 2.0, 3.0, edgecolor=CBND,
            facecolor=CFEAS, zorder=0, label='Feasible $X_1$')
52      ax.add_patch(feas)
53      ax.contour(X1G, X2G, ZQ1, levels=20, zorder=1)
54      ax.plot(hist_X1[:,0], hist_X1[:,1], '-o', ms=3, zorder=2, label='Trajectory')
55      ax.plot(*x0_q1, 's', ms=8, label='$x^{(0)}$')
56      ax.plot(*hist_X1[-1], 'o', ms=12, label='Final iterate')
57      ax.set_xlim(0.0, 3.0); ax.set_ylim(0.0, 4.0)
58      ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
59      ax.set_title('Q1: (d-I) Contour + Trajectory - $X_1$')
60      ax.legend(fontsize=8)
61
62      ax = axes[1]
63      ax.semilogy(func_hist_X1, label='$f(x^{(t)})$')
64      ax.set_xlabel('Iteration'); ax.set_ylabel('$f(x^{(t)})$ [log scale]')
65      ax.set_title('(Q1: d-II) Loss vs Iteration - $X_1$')
66      ax.grid(True, alpha=0.3); ax.legend()
67
68      ax = axes[2]
69      ax.plot(hist_X1[:,0], lw=1.6, label='$x_1^{(t)}$')
70      ax.plot(hist_X1[:,1], lw=1.6, linestyle='--', label='$x_2^{(t)}$')
71      ax.set_xlabel('Iteration'); ax.set_ylabel('Coordinate value')
72      ax.set_title('Q1: (d-III) Coordinates vs Iteration - $X_1$')
73      ax.legend(); ax.grid(True, alpha=0.3)
74
75      plt.tight_layout()
76      plt.savefig('q1d_X1.png', dpi=150, bbox_inches='tight')
77      plt.show()
78      plt.close()
79
80      fig, axes = plt.subplots(1, 3, figsize=(16, 5))
81
82      from matplotlib.patches import Polygon as MplPolygon
83      verts_X2 = np.array([[0.5,0.5],[3.0,3.0],[0.5,3.0]])
84      ax = axes[0]
85      feas2 = MplPolygon(verts_X2, closed=True, zorder=0, edgecolor=CBND, facecolor
            =CFEAS, label='Feasible $X_2$')
86      ax.add_patch(feas2)
87      ax.contour(X1G, X2G, ZQ1, levels=20, zorder=1)
88      ax.plot(hist_X2[:,0], hist_X2[:,1], '-o', color=C1, ms=3, zorder=2, label='
            Trajectory')
89      ax.plot(*x0_q1, 's',  ms=8, label='$x^{(0)}$')
90      ax.plot(*hist_X2[-1], 'o', ms=12, label='Final iterate')
91      ax.set_xlim(0.0, 3.0); ax.set_ylim(0.0, 4.0)
92      ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
93      ax.set_title('Q1: (e-I) Contour + Trajectory - $X_2$')
```

```
    ax.legend(fontsize=8)

    ax = axes[1]
    ax.semilogy(func_hist_X2, lw=1.8, label='$f(x^{(t)})$')
    ax.set_xlabel('Iteration'); ax.set_ylabel('$f(x^{(t)})$ [log scale]')
    ax.set_title('Q1: (e-II) Loss vs Iteration - $X_2$')
    ax.grid(True, alpha=0.3); ax.legend()

    ax = axes[2]
    ax.plot(hist_X2[:,0], lw=1.6, label='$x_1^{(t)}$')
    ax.plot(hist_X2[:,1], lw=1.6, linestyle='--', label='$x_2^{(t)}$')
    ax.set_xlabel('Iteration'); ax.set_ylabel('Coordinate value')
    ax.set_title('Q1: (e-III) Coordinates vs Iteration - $X_2$')
    ax.legend(); ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('q1e_X2.png', dpi=150, bbox_inches='tight')
    plt.show()
    plt.close()
```

A.2 Q2 - Penalty Methods and Primal-Dual

```
    def func_q2(x1, x2):
        return (x1 - 0.2)**2 + (x2 - 2)**2

    def grad_q2(x1, x2):
        return np.array([2*(x1 - 0.2), 2*(x2 - 2)])

    def g1(x1, x2): return 0.5 - x1
    def g2(x1, x2): return 1.0 - x1*x2

    def grad_g1(x1, x2): return np.array([-1.0, 0.0])
    def grad_g2(x1, x2): return np.array([-x2, -x1])

    def F_penalty(x1, x2, lam1, lam2):
        return func_q2(x1,x2) + lam1*max(0,g1(x1,x2)) + lam2*max(0,g2(x1,x2))

    def compute_grad_F(x1, x2, lam1, lam2):
        gf = grad_q2(x1, x2)
        gp = np.zeros(2)
        if g1(x1,x2) > 0:
            gp += lam1 * grad_g1(x1,x2)
        if g2(x1,x2) > 0:
            gp += lam2 * grad_g2(x1,x2)
        return gf + gp

    x0_q2 = np.array([1.4, 0.6])
    alpha_q2 = 0.01
    n_q2 = 300

    def compute_gd_penalty_q2(x0, lam1, lam2, alpha, itr):
        x = x0.copy().astype(float)
        hist = [x.copy()]
        g1_hist, g2_hist = [g1(*x)], [g2(*x)]
        for _ in range(itr):
```

```
            g = compute_grad_F(*x, lam1, lam2)
            x = x - alpha * g
            hist.append(x.copy())
            g1_hist.append(g1(*x))
            g2_hist.append(g2(*x))
        return np.array(hist), np.array(g1_hist), np.array(g2_hist)

hist_small_q2, g1_small, g2_small = compute_gd_penalty_q2(x0_q2, 0.5, 0.5,
    alpha_q2, n_q2)
hist_large_q2, g1_large, g2_large = compute_gd_penalty_q2(x0_q2, 4.0, 4.0,
    alpha_q2, n_q2)

def primal_dual(x0, alpha=0.06, beta=0.08, n_iters=150):
    x = x0.copy().astype(float)
    lam = np.array([0.0, 0.0])
    hist_x = [x.copy()]
    hist_lam = [lam.copy()]
    for _ in range(n_iters):
        gf = grad_q2(*x)
        gcons = lam[0]*grad_g1(*x) + lam[1]*grad_g2(*x)
        x = x - alpha*(gf + gcons)
        lam[0] = max(0, lam[0] + beta*g1(*x))
        lam[1] = max(0, lam[1] + beta*g2(*x))
        hist_x.append(x.copy())
        hist_lam.append(lam.copy())
    return np.array(hist_x), np.array(hist_lam)

hist_pd, hist_lam = primal_dual(x0_q2, alpha=0.06, beta=0.08, n_iters=150)

x1g2 = np.linspace(0.1, 2.5, 300)
x2g2 = np.linspace(0.1, 3.5, 300)
X1G2, X2G2 = np.meshgrid(x1g2, x2g2)
ZQ2 = func_q2(X1G2, X2G2)

feas_mask = (X1G2 >= 0.5) & (X1G2*X2G2 >= 1.0)

def plot_feasible_q2(ax):
    ax.contourf(X1G2, X2G2, feas_mask.astype(float), levels=[0.5,1.5],
                colors=[CFEAS], alpha=0.6, zorder=0)
    xx = np.linspace(0.5, 2.5, 300)
    ax.plot(xx, 1.0/xx, color=CBND, lw=1.2, zorder=1, label='$x_1 x_2=1$')
    ax.axvline(0.5, color=CBND, lw=1.2, linestyle='--', label='$x_1=0.5$')

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

ax = axes[0]
plot_feasible_q2(ax)
ax.contour(X1G2, X2G2, ZQ2, levels=15, zorder=1)
ax.plot(hist_small_q2[:,0], hist_small_q2[:,1], '-o', ms=2, label='Trajectory
    (small $\lambda$)')
ax.plot(*x0_q2, 's', ms=8, label='$x^{(0)}$')
ax.set_xlim(0.1,2.5); ax.set_ylim(0.1,3.5)
ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
ax.set_title('Q2: (d-I) Small penalty ($\lambda=0.5$)')
ax.legend(fontsize=8)
```

```
86
87      ax = axes[1]
88      plot_feasible_q2(ax)
89      ax.contour(X1G2, X2G2, ZQ2, levels=15, linewidths=0.7, zorder=1)
90      ax.plot(hist_large_q2[:,0], hist_large_q2[:,1], '-o', ms=2, lw=1.3, label='
            Trajectory (large $\lambda$)')
91      ax.plot(*x0_q2, 's', ms=8, label='$x^{(0)}$')
92      ax.set_xlim(0.1,2.5); ax.set_ylim(0.1,3.5)
93      ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
94      ax.set_title('Q2: (d-II) Large penalty ($\lambda=4.0$)')
95      ax.legend(fontsize=8)
96
97      ax = axes[2]
98      itr_q2 = np.arange(n_q2+1)
99      ax.plot(itr_q2, g1_small, lw=1.6, label='$g_1$ small $\lambda$')
100     ax.plot(itr_q2, g2_small, lw=1.6, linestyle='--', label='$g_2$ small $\lambda
            $')
101     ax.plot(itr_q2, g1_large, lw=1.6, label='$g_1$ large $\lambda$')
102     ax.plot(itr_q2, g2_large, lw=1.6, linestyle='--', label='$g_2$ large $\lambda
            $')
103     ax.axhline(0, color='#888888', lw=0.8, linestyle=':')
104     ax.set_xlabel('Iteration'); ax.set_ylabel('Constraint value')
105     ax.set_title('Q2: (d-III) Constraint violations vs Iteration')
106     ax.legend(fontsize=8); ax.grid(True, alpha=0.3)
107
108     plt.tight_layout()
109     plt.savefig('q2d_penalty.png', dpi=150, bbox_inches='tight')
110     plt.show()
111     plt.close()
112
113     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
114
115     ax = axes[0]
116     plot_feasible_q2(ax)
117     ax.contour(X1G2, X2G2, ZQ2, levels=15, zorder=1)
118     ax.plot(hist_pd[:,0], hist_pd[:,1], '-o', ms=2, label='Primal-dual trajectory
            ')
119     ax.plot(*x0_q2, 's', ms=8, label='$x^{(0)}$')
120     ax.set_xlim(0.1,2.5); ax.set_ylim(0.1,3.5)
121     ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
122     ax.set_title('Q2: (e-I) Primal-dual contour + trajectory')
123     ax.legend(fontsize=8)
124
125     ax = axes[1]
126     ax.plot(hist_pd[:,0], label='$x_1^{(t)}$')
127     ax.plot(hist_pd[:,1], linestyle='--', label='$x_2^{(t)}$')
128     ax.set_xlabel('Iteration'); ax.set_ylabel('Coordinate')
129     ax.set_title('Q2: (e-II) $x_1, x_2$ vs Iteration - Primal-dual')
130     ax.legend(); ax.grid(True, alpha=0.3)
131
132     ax = axes[2]
133     ax.plot(hist_lam[:,0], label='$\lambda_1^{(t)}$')
134     ax.plot(hist_lam[:,1], linestyle='--', label='$\lambda_2^{(t)}$')
135     ax.set_xlabel('Iteration'); ax.set_ylabel('Multiplier value')
136     ax.set_title('Q2: (e-III) $\lambda_1, \lambda_2$ vs Iteration')
```

```
    ax.legend(); ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('q2e_primaldual.png', dpi=150, bbox_inches='tight')
    plt.show()
    plt.close()
```

A.3 Q3 - Frank-Wolfe Algorithm

```
    def f_q3(x1, x2):
        return (x1 - 1.5)**2 + (x2 - 1.2)**2

    def grad_q3(x1, x2):
        return np.array([2*(x1 - 1.5), 2*(x2 - 1.2)])

    VERTICES = np.array([
        [0.5, 0.5],
        [3.0, 0.5],
        [3.0, 1.0],    # x1=3, x1+x2=4 => x2=1
        [1.0, 3.0],    # x2=3, x1+x2=4 => x1=1
        [0.5, 3.0],
    ])

    def frank_wolfe_lp(grad, vertices):
        scores = vertices @ grad
        return vertices[np.argmin(scores)]

    def calc_frank_wolfe(x0, grad_fn, vertices, beta, itrs):
        x = x0.copy().astype(float)
        hist_x = [x.copy()]
        hist_z = []
        hist_f = [f_q3(*x)]
        for t in range(itrs):
            g = grad_fn(*x)
            z = frank_wolfe_lp(g, vertices)
            hist_z.append(z.copy())
            gamma = 2.0 / (t + 2)
            x = (1 - beta)*x + beta*z
            hist_x.append(x.copy())
            hist_f.append(f_q3(*x))
        return np.array(hist_x), np.array(hist_f), np.array(hist_z)

    x0_q3 = np.array([2.8, 0.8])
    n_q3 = 80

    hist_fw08_q3, fv_fw08_q3, hz_fw08_q3 = calc_frank_wolfe(x0_q3, grad_q3,
       VERTICES, beta=0.8, itrs=n_q3)
    hist_fw095_q3, fv_fw095_q3, hz_fw095_q3 = calc_frank_wolfe(x0_q3, grad_q3,
       VERTICES, beta=0.95, itrs=n_q3)

    def project_X3(x):
        for _ in range(50):
            x[0] = np.clip(x[0], 0.5, 3.0)
            x[1] = np.clip(x[1], 0.5, 3.0)
            if x[0] + x[1] > 4.0:
```

```
                excess = x[0] + x[1] - 4.0
                x[0] -= excess/2; x[1] -= excess/2
        return x

    hist_pgd3 = pgd(x0_q3, grad_q3, project_X3, alpha=0.05, itr=n_q3)
    fv_pgd3 = np.array([f_q3(*p) for p in hist_pgd3])

    x1g3 = np.linspace(0.0, 3.5, 300)
    x2g3 = np.linspace(0.0, 3.5, 300)
    X1G3, X2G3 = np.meshgrid(x1g3, x2g3)
    ZQ3 = f_q3(X1G3, X2G3)
    feas3 = (X1G3>=0.5)&(X2G3>=0.5)&(X1G3+X2G3<=4)&(X1G3<=3)&(X2G3<=3)

    def draw_feasible_X3(ax):
        ax.contourf(X1G3, X2G3, feas3.astype(float), levels=[0.5,1.5],
                    colors=[CFEAS], alpha=0.5, zorder=0)
        vx = np.append(VERTICES[:,0], VERTICES[0,0])
        vy = np.append(VERTICES[:,1], VERTICES[0,1])
        ax.plot(vx, vy, color=CBND, lw=1.2, zorder=1)
        ax.scatter(VERTICES[:,0], VERTICES[:,1], color=C4, s=50, zorder=3, label=
            'Vertices')

    for bval, hist_fw, fv_fw, hz_fw, tag in [
        (0.8, hist_fw08_q3, fv_fw08_q3, hz_fw08_q3, '08'),
        (0.95, hist_fw095_q3, fv_fw095_q3, hz_fw095_q3, '095')]:

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    ax = axes[0]
    draw_feasible_X3(ax)
    ax.contour(X1G3, X2G3, ZQ3, levels=15, zorder=1)
    ax.plot(hist_fw[:,0], hist_fw[:,1], '-o', ms=3, label='FW trajectory')
    ax.plot(*x0_q3, 's', ms=8, label='$x^{(0)}$')
    ax.plot(*hist_fw[-1], '*', ms=12, label='Final')
    ax.set_xlim(0,3.5); ax.set_ylim(0,3.5)
    ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
    ax.set_title(f'Q3: (d-I) FW Contour + Trajectory ($\\beta={bval}$)')
    ax.legend(fontsize=8)

    ax = axes[1]
    ax.semilogy(fv_fw, label='$f(x^{(t)})$')
    ax.set_xlabel('Iteration'); ax.set_ylabel('$f$ [log]')
    ax.set_title(f'Q3: (d-II) $f(x^{{(t)}})$ vs Iteration ($\\beta={bval}$)')
    ax.legend(); ax.grid(True, alpha=0.3)

    ax = axes[2]
    ax.plot(hist_fw[:,0], label='$x_1^{(t)}$')
    ax.plot(hist_fw[:,1], linestyle='--', label='$x_2^{(t)}$')
    if len(hz_fw) > 0:
        ax.plot(np.arange(1, len(hz_fw)+1), hz_fw[:,0], linestyle=':', label='$z_
            1^{(t)}$')
        ax.plot(np.arange(1, len(hz_fw)+1), hz_fw[:,1], linestyle='-.', label='$z
            _2^{(t)}$')
    ax.set_xlabel('Iteration'); ax.set_ylabel('Coordinate')
    ax.set_title(f'Q3: (d-III) $x^{{(t)}}$ and $z^{{(t)}}$ ($\\beta={bval}$)')
```

```
    ax.legend(fontsize=8); ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(f'q3d_fw{tag}.png', dpi=150, bbox_inches='tight')
    plt.show()
    plt.close()

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    ax = axes[0]
    ax.semilogy(fv_fw08_q3, label='FW $\\beta=0.8$')
    ax.semilogy(fv_fw095_q3, linestyle='--', label='FW $\\beta=0.95$')
    ax.semilogy(fv_pgd3, linestyle=':', label='PGD $\\alpha=0.05$')
    ax.set_xlabel('Iteration'); ax.set_ylabel('$f$ [log]')
    ax.set_title('Q3: (e) FW vs PGD: $f(x^{(t)})$')
    ax.legend(); ax.grid(True, alpha=0.3)

    ax = axes[1]
    draw_feasible_X3(ax)
    ax.contour(X1G3, X2G3, ZQ3, levels=15, linewidths=0.7, zorder=1)
    ax.plot(hist_fw08_q3[:,0], hist_fw08_q3[:,1], '-', label='FW $\\beta=0.8$')
    ax.plot(hist_fw095_q3[:,0], hist_fw095_q3[:,1], '--', label='FW $\\beta=0.95$
        ')
    ax.plot(hist_pgd3[:,0], hist_pgd3[:,1], ':', label='PGD')
    ax.plot(*x0_q3, 's', ms=8, label='$x^{(0)}$')
    ax.set_xlim(0,3.5); ax.set_ylim(0,3.5)
    ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
    ax.set_title('Q3: (e) Trajectory comparison')
    ax.legend(fontsize=8)

    plt.tight_layout()
    plt.savefig('q3e_comparison.png', dpi=150, bbox_inches='tight')
    plt.show()
    plt.close()
```